Organiser le delivery logiciel AI-native avec Gas Town

Dev With AI Live #4

Jean-Louis Rigau & Emmanuel Sciara

Vendredi 12 juin 2026

#Engineering AI-native #Delivery distribué #Orchestration d'agents #Exécution supervisée

Ouvrir la présentation

Intervenants



INTERVENANT

Jean-Louis Rigau

Vibe coder & AI-native advisor. J’aide les organisations et les équipes engineering à faire évoluer leurs plateformes et leurs workflows de delivery pour intégrer durablement les systèmes multi-agents et les pratiques agentiques.

linkedin.com/in/jlrigau



INTERVENANT

Emmanuel Sciara

Agent Lead & ingénieur AI-native. J'interviens sur la stratégie d'entreprise, le déploiement et le coaching de l'utilisation des coding agents, avec une spécialisation Claude Code.

linkedin.com/in/emmanuelsciara

Les 8 étapes du développement IA

1



IA nulle ou quasi nulle

2



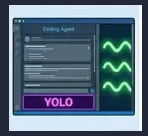
Assistant IDE avec permissions

3



Agent IDE en mode YOLO

4



Agent IDE en mode large

5



Agent CLI en session unique

6



Agent CLI en multi-sessions

7



10+ agents, pilotés à la main

8



Construire un orchestrateur

CHANGEMENT DE PRATIQUE

D'une requête à un assistant vers la gestion d'un système de delivery.

- Les premières étapes améliorent le flux individuel.
- Les étapes intermédiaires augmentent le débit (CLI).
- Les dernières étapes créent un enjeu de coordination.
- Gas Town entre en jeu quand le pilotage manuel n'est plus gérable.

TAKEAWAY
Le développement avec l'IA évolue de l'assistance individuelle vers la coordination organisationnelle.

Steve Yegge



Photo : Bram Cymet, CC BY-NC 2.0

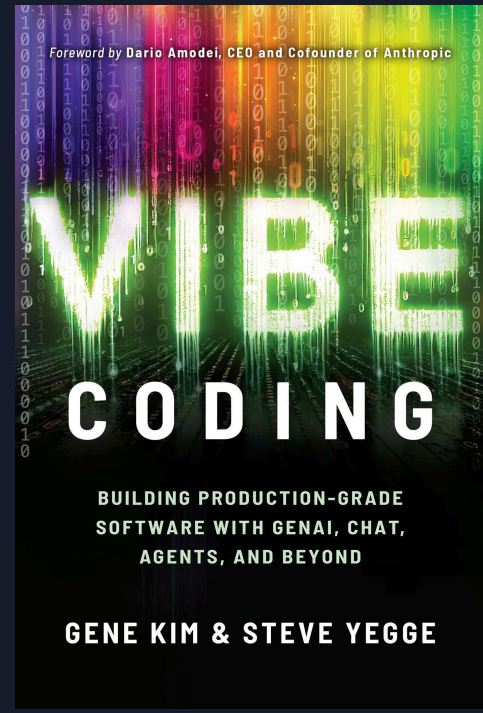
DE LA PRATIQUE À L'ORGANISATION

Steve Yegge cadre la bascule de l'utilisation individuelle des agents de code au delivery logiciel AI-native.

Vibe Coding
Developpement de logiciel de qualité production avec des agents de coding IA.

Gas Town
Un modèle opérationnel pour coordonner de nombreux agents sur des projets réels.

Beads
Un registre de travail basé sur Git pour workflows d'ingénierie agentiques.



Couverture : IT Revolution / Simon & Schuster

Gas Town : un système de delivery urbanisé

TOWN

Quartier général agentique

Un espace opérationnel séparé où vivent orchestration, identité, mail et traçage du travail.

MAYOR

Coordination

C'est le point d'entrée : absorbe le bruit des workers et transforme l'intention en travail coordonné.

RIGS (TOUR DE FORAGE)

Usines projets

Polecats, Crew, Witness et Refinery opèrent directement sur des repositories.

LA VILLE COMME MODÈLE OPÉRATIONNEL

Gas Town rend le vibe coding opérationnel sans nier le chaos.

- Les agents deviennent des workers visibles, avec une identité.
- Les beads fournissent un registre du travail, de la mémoire, du mail et de la provenance, avec Git comme backend.
- Les convoys suivent le delivery ; les Polecats exécutent.
- Witness et Refinery évitent le chaos du merge.

TAKEAWAY

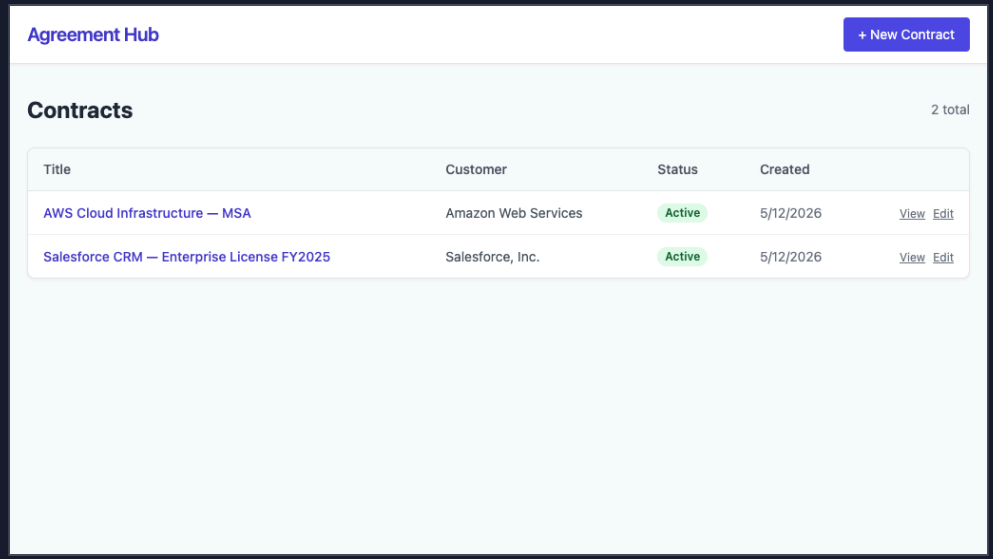
Gas Town rend l'utilisation des agents de code observable et coordonnée.

Application de démo

AGREEMENT HUB

Application CRUD de Contract Lifecycle Management servant de cible de delivery réaliste.

- Stack : frontend React/Vite, backend Express/TypeScript et base SQLite.
- Gas Town travaille sur un repository forké, donc la démo repart d'une baseline propre.
- La démo est réelle : le résultat peut différer du plan.



DÉROULÉ DÉMO

Démarrer la boucle : ouvrir Agreement Hub → montrer l'état courant → demander au Mayor.

À partir d'ici, chaque signal live des workers indique ce qu'il faut observer.

Copier le prompt **Ouvrir l'app ↗**

Pourquoi les workflows single-agent cassent

AGENT UNIQUE

Très bon assistant local

Un seul contexte, un seul fil d'exécution, une seule branche de travail.

01

Saturation du contexte

Les contraintes importantes disparaissent la session grandit.

02

Exécution séquentielle

Le débit est limité par une seule boucle de worker.

03

Pas de mécanisme de coordination

Les dépendances et les responsabilités restent implicites.

04

Pression sur le merge

Les changements parallèles deviennent difficiles à intégrer proprement.

05

Pas de supervision opérationnelle

Le travail bloqué devient difficile à observer et à récupérer.

PROBLÈME

le développement avec IA se fait localement, alors que le delivery logiciel est organisationnel.

- Les Context Windows ne modélisent pas les dépendances.
- Le travail séquentiel limite le débit.
- Le travail parallèle crée une pression sur l'intégration.
- La supervision devient un sujet de premier ordre.

TAKEAWAY

Le delivery logiciel est organisationnel. Le goulot devient la coordination.

Gas Town organise le travail, pas les prompts

ENTRÉE

Intention humaine

Une demande en langage naturel, incomplète par construction.

COUCHE D'ORGANISATION

Gas Town structure le delivery

- **Mayor** planifie et coordonne.
- **Beads** rendent le travail adressable.
- **Convoys** suivent les flux de delivery.
- **Witness & Refinery** gardent l'exécution observable.
- **Polecats** font le travail.

SORTIE

Delivery observable

Statuts, dépendances, responsabilités, progression et flux de merge deviennent inspectables.

RÉPONSE

Gas Town transforme l'intention en objets de delivery partagés.

- Le Mayor sépare planification et exécution.
- Beads et convoys rendent le travail adressable et traçable.
- Witness et Refinery rendent la supervision et la convergence visibles.
- Les Polecats exécutent.

TAKEAWAY

Le passage à l'échelle vient de la coordination partagée, pas de prompts plus gros.

LIVE SIGNAL Le Mayor transforme une demande en beads, convoys, affectations et statuts visibles.

L'organisation

TOWN

🧿 **Overseer** Intention et contrôle humains	🏛️ **Mayor** Coordination cross-rig	🐴 **Deacon** Santé système

RIG

Un repository projet avec son propre environnement d'exécution.

👥 **Crew** Workers persistants	🐱 **Polecats** Workers éphémères	🦉 **Witness** Supervision du rig	🏭 **Refinery** Queue de merge

POURQUOI C'EST IMPORTANT

Gas Town s'articule autour d'une town et de ses rigs.

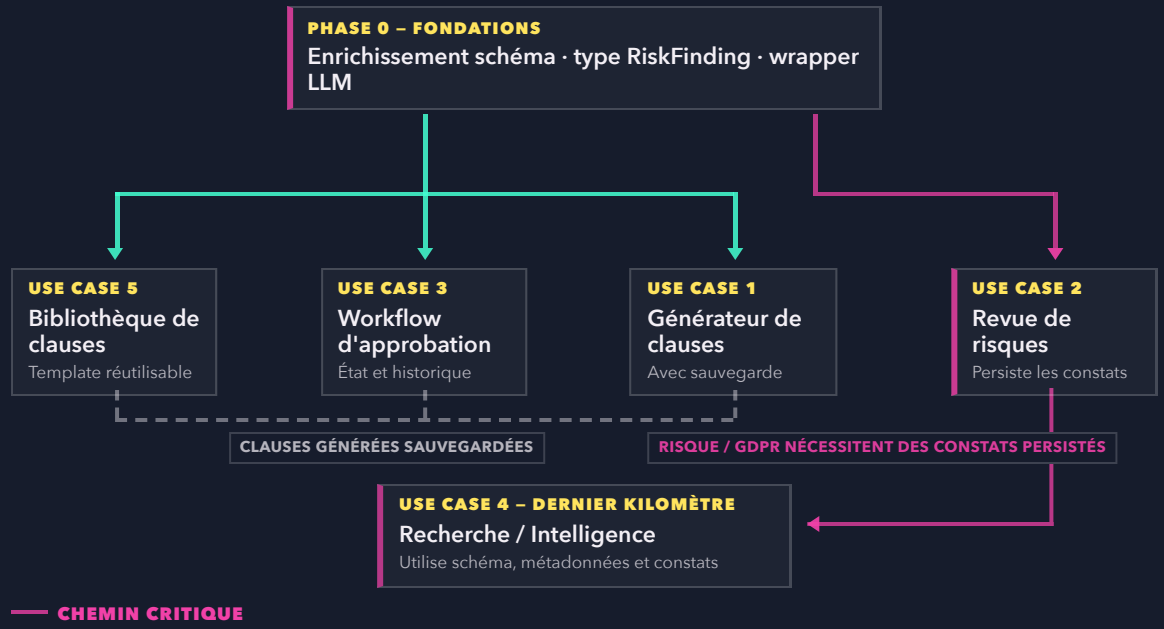
- La Town coordonne les repositories.
- Chaque Rig cible une vraie codebase.
- Les Crew sont persistants; les Polecats sont des workers d'exécution jetables.
- Witness et Refinery opèrent au niveau du rig.

TAKEAWAY

Gas Town sépare coordination, exécution, supervision et intégration.

LIVE SIGNAL Identifier le rig, le crew persistant et les workers jetables attachés au repository.

Beads : découpage du travail & dépendances



POURQUOI C'EST IMPORTANT

Les beads transforment un besoin en travail exécutable, avec ordre, responsabilités et statut.

- Les dépendances exposent le chemin critique.
- Le travail qui est prêt peut être exécuté immédiatement.
- Le travail bloqué devient visible au lieu de rester implicite.
- Les hooks gardent les workers connectés aux affectations actives.

TAKEAWAY
Les beads montrent ce qui peut démarrer maintenant, ce qui est bloqué, et pourquoi.

LIVE SIGNAL Création des beads, statut ready/blocked, graphe de dépendances et hook courant.

Convoys : flux de delivery

CONVOY PHASE 0 — Étape 0
Fondations
socle partagé · débloque les Use Cases 1-5

CONVOY USE CASE 5 — Étape 1
Bibliothèque de clauses
flux CRUD · clauses réutilisables

CONVOY USE CASE 3 — Étape 1
Workflow d'approbation
state machine · historique

CONVOY USE CASE 1 — Étape 2
Générateur de clauses
flux IA · sauvegarde en bibliothèque

CONVOY USE CASE 2 — Étape 2
Revue de risques
flux IA · constats persistés

CONVOY USE CASE 4 — Étape 3
Recherche / Intelligence
dernier kilomètre · dépend du schéma et des constats de risque

POURQUOI C'EST IMPORTANT

Les convoys rendent chaque use case visible comme flux de delivery.

- Un même numéro d'étape signifie du travail parallèle.
- La progression devient visible entre workers.
- Référence partagée entre les rôles.

TAKEAWAY
Les convoys transforment le graphe de beads en flux de delivery observables.

LIVE SIGNAL Création des convoys, statut des convoys et notification de livraison.

Polecats : exécution parallèle

POLECATS	BEADS
🐱 FURIOSA	Fondations Bibliothèque de clauses Revue de risques
🐱 NUX	Workflow d'approbation contrat
🐱 SLIT	Générateur de clauses Recherche / Intelligence MVP

UNE DEMANDE PRODUIT

↓

BEADS CADRÉS

↓

WORKERS D'EXÉCUTION ISOLÉS (POLECATS)

↓

DELIVERY COORDONNÉ

POURQUOI C'EST IMPORTANT

Le Mayor assigne des beads cadrés à des Polecats isolés, chacun dans son contexte et sa branche.

- Lancés à la demande.
- Les affectations de travail restent isolées entre elles.
- Les Polecats disparaissent après livraison.
- La parallélisation augmente le débit, pas le chaos.

TAKEAWAY
Les Polecats scalent l'exécution via des contextes isolés.

LIVE SIGNAL Plusieurs Polecats, branches, commits et statuts de delivery en parallèle.

La supervision garde l'exécution opérationnelle



POURQUOI C'EST IMPORTANT

La supervision Gas Town est organisée en couches, pas autour d'un watcher unique.

- Le Daemon réveille le Boot pour vérifier la santé.
- Le Deacon garde la town en bonne santé.
- Le Witness maintient l'exécution du rig.
- la Refinery maintient la convergence.

TAKEAWAY
La supervision rend l'exécution distribuée opérationnelle.

LIVE SIGNAL Santé du Deacon, statut du Witness, reprise des Polecats, Refinery queue et convergence du delivery.

Vérification Agreement Hub

INSPECTION LOCALE

Nous inspectons le repository livré en lançant Agreement Hub localement et en le comparant à la version de départ.

- L'application n'est pas déployée pendant la démo.
- Nous clonons le repository complet produit par Gas Town.
- Nous lançons ce repository localement, puis le comparons à la baseline.

Que va révéler l'application livrée?

USE CASES IMPLÉMENTÉS

Générateur de clauses	☑
Revue de risques	☑
Workflow d'approbation	☐
Recherche & Intelligence	☐
Bibliothèque de clauses	☐

CHECKPOINT DE VÉRIFICATION

La boucle de delivery Gas Town s'arrête ici.

Nous clonons le repository livré, lançons Agreement Hub localement, et le comparons à la version de départ.

Ouvrir l'app ↗

Ce que Gas Town change

	PROBLÈME **Les workflows single-agent cassent quand le développement devient un enjeu de delivery.**	RÉPONSE GAS TOWN **Transformer le delivery en modèle opérationnel distribué et supervisé.**
01	**Saturation du contexte** Les contraintes importantes disparaissent la session grandit.	Structurer le travail en beads, artefacts et passages de relais explicites.
02	**Exécution séquentielle** Le débit est limité par une seule boucle de worker.	Utiliser les Polecats pour exécuter le travail indépendant en parallèle.
03	**Pas de mécanisme de coordination** Les dépendances et les responsabilités restent implicites.	Mayor, Crew et Convoys rendent responsabilités et progression visibles.
04	**Pression sur le merge** Les changements parallèles deviennent difficiles à intégrer proprement.	La Refinery gère la convergence au lieu de laisser l'intégration au hasard.
05	**Pas de supervision opérationnelle** Le travail bloqué devient difficile à observer et à récupérer.	Witness, Deacon et les boucles de récupération gardent l'exécution observable.

Merci

Questions & discussion

EXEMPLES DE QUESTIONS

Combien coûte le delivery d'une feature avec Gas Town?

Quel niveau de contrôle avons-nous sur les modèles?

Quand faut-il utiliser le Crew plutôt que seulement le Mayor?